KV Cache 前置知识

> 这份文档面向已读完 LLM 大模型前置知识的读者。如果你还不太清楚 Q/K/V、Attention、Prefill/Decode，建议先回顾 LLM大模型前置知识。

> 🎯 **读完这篇你能**：理解 KV Cache 省了什么、为什么省、怎么省，能说出 MQA / GQA / MLA / PagedAttention 各自做了什么，能判断不同优化方案的适用场景。

一、KV Cache 到底是什么?

一句话：把生成过程中每个 token 投影好的 K 和 V 存起来，后面生成新 token 时直接拿来用，不用重新投影。

回想 LLM 前置知识里的流程：每个 token 进来，要经过 W_K、W_V 两套权重矩阵投影，才产生 K 和 V。如果不缓存，生成第 5 个 token 时，前 4 个 token 的 K、V 要全部重新算一遍——这 4 个 token 已经被投影过 4 次了。

KV Cache 的直觉类比：**考试时，每道题都先做一遍，后面检查时只核对新的部分，旧题不用重做。**

它不是一篇论文提出的，而是 HuggingFace 团队在 2019 年给 GPT-2 做推理时加的一个工程优化——在代码里加了个叫 `past_key_values` 的参数，把算好的 K、V 存下来。后面所有人觉得"这个好用"，就成了标准。

二、KV Cache 有多大?

一个 token 的 KV Cache 大小取决于模型的维度配置。公式：

```
每 token KV Cache 大小 = 2 × 层数 × 头数 × 每头维度 × 精度字节数
```

举个例子——Llama 2 7B（32 层、32 头、128 维度、FP16）：

```
每 token KV Cache = 2 × 32 × 32 × 128 × 2 字节 ≈ 0.5 MB
1000 token 序列的 KV Cache ≈ 0.5 GB
```

注意这只是**一个请求**的缓存。生产环境并发 100 个请求、每个 4000 token，单是 KV Cache 就能吃掉几十 GB 显存。这就是为什么 KV Cache 优化如此关键——不是锦上添花，是能跑和不能跑的区别。

三、KV Cache 的核心矛盾

```
要保留所有 token 的 KV → 显存放不下
丢弃一些 KV → 可能丢掉关键信息，答案变差
```

所有 KV Cache 优化都在这个张力之间找平衡。不同方案从不同角度切入。

四、优化的四条路

路线一：减少每个 token 的 KV 头数（架构层）

标准 Multi-Head Attention（MHA）中，每个注意力头有自己的一套 K 和 V。比如 Llama 2 7B 有 32 个头，就存 32 套 KV。

MQA（Multi-Query Attention，2019）：所有 Q 头共享同一套 K、V。32 套变 1 套，KV Cache 直接缩小 32 倍。代价是注意力"表达能力"下降，某些任务质量有损失。

GQA（Grouped-Query Attention，2023）：不搞全共享，搞分组。比如 32 个 Q 头分 8 组，每组共享一套 KV。KV Cache 缩小 4 倍（32/8），质量几乎无损。这是 2024-2026 年最主流的方案——Llama 2/3/4、Mistral、Qwen 全部采用。

打个比方：

- MHA = 32 个人各自拿自己的笔记本（KV），占地方
- MQA = 所有人共看一块白板（KV），省地方但看不太清
- GQA = 分 8 组，每组一块白板——够省也够用

路线二：别连续存，分页管理（系统层）

传统 KV Cache 像停车场——每个序列来了先划一块连续的车位给它。问题是：序列长度未知，预先分配大量连续显存，用不完的就浪费了，实际利用率只有 20-40%。

PagedAttention（vLLM，2023） 把操作系统"虚拟内存分页"的思想搬过来：KV Cache 切成固定大小的块（block），存在哪都行，用的时候通过映射表找到。内存浪费从 60-80% 降到 4% 以下。

直觉类比：停车场变成了共享单车——车停在哪都行，扫个码就能找到。不用给每个用户预留固定车位。

RadixAttention（SGLang，2024） 更进一步——用基数树管理 prefix 共享。多个请求的相同前缀（比如同一个 system prompt）在树里自动合并存储，不重复缓存。

路线三：挑重要的 token 留，不重要的丢弃（序列维）

不是所有 token 对生成的贡献都一样大。有些 token（高频词、虚词）对注意力贡献极小。

StreamingLLM（2023）：发现模型对前 4 个 token 分配了不成比例的高注意力（不管它们语义上重不重要），这 4 个叫"Attention Sink"。方案：保留前 4 个 + 最近的 N 个，中间的全扔掉。KV Cache 大小固定，可以无限长度流式生成。

H2O（2023）：发现约 20% 的 token 贡献了 80% 的注意力分数，这些叫"Heavy Hitters"。保留它们 + 最近的 token，中间淘汰。

SnapKV（2024）：生成前，用 prompt 末尾几个 token 的注意力模式"投票"选出哪些 KV 重要，一次性压缩好，生成阶段不额外消耗。

路线四：少存几个比特（数值维）

不减少 token 数，而是每个 K、V 用更少的比特存。

KVQuant（2024）：把 16-bit 的 K、V 压缩到 3-4 bit，几乎无损（perplexity 下降不到 0.1）。

KIVI（2024）：压到 2 bit，不需要额外校准数据。

直觉类比：照片从 RAW 格式压成 JPEG——肉眼基本看不出差别，但文件大小差了一个数量级。

五、MLA — 范式级突破

2024 年 DeepSeek-V2 提出的 **Multi-head Latent Attention（MLA）** 从数学层面重新定义了压缩方式。

前面 MQA/GQA 是在注意力头数量上做文章。MLA 换了一个问题：**能不能不存完整的 K 和 V，而是存一个低维压缩表示，用的时候展开？**

实现方式：K 和 V 先投影到一个很小的"潜在空间"，缓存这个潜在向量。注意力计算时，通过上投影矩阵"解压"出等效结果。因为压缩-解压矩阵可以被吸收进 Q 的投影矩阵中，解码时根本不需要显式重建完整 K、V。

效果：KV Cache 减少约 93%，吞吐量提升至原来的 5.76 倍。

如果是 MQA 是"共用笔记本"，MLA 就是"用速记符号写下来，要用的时候脑子自动展开"。

六、这些方案能叠加吗？

能，而且实际生产环境就是这么干的。四类方案可以组合：

```
GQA（减少 KV 头数）
  + PagedAttention（分页管理）
    + SnapKV（挑重要的留）
      + KVQuant（4-bit 量化）
```

vLLM 和 SGLang 都已支持多方案组合。组合效果近似倍增。

七、一张图总结

```
KV Cache 优化方案的进化树

MHA（每头独立 KV，标准配置）
 │
 ├──→ 架构层 ──→ MQA（2019, 1套KV）──→ GQA（2023, 分组KV）──→ MLA（2024, 低秩压缩）
 │      减少KV头数                 折中方案                 不存完整KV
 │
 ├──→ 系统层 ──→ PagedAttention（2023, 分页管理）──→ RadixAttention（2024, 树形共享）
 │      消除内存碎片                          前缀自动去重
 │
 ├──→ 序列维 ──→ H2O（2023, 留注意力高分token）──→ SnapKV（2024, 一次投票定稿）
 │      淘汰不重要的                            生成前压缩
 │
 └──→ 数值维 ──→ KVQuant（2024, 3-bit）──→ KIVI（2024, 2-bit）
        少存比特                    更少比特
```

参考来源

- **Fast Transformer Decoding: One Write-Head is All You Need** (Shazeer, 2019) — 提出 MQA，首个针对 KV Cache 大小的架构优化 — https://arxiv.org/abs/1911.02150
- **GQA: Training Generalized Multi-Query Transformer Models** (Ainslie et al., Google, 2023) — 提出 GQA，MHA 与 MQA 的折中方案 — https://arxiv.org/abs/2305.13245
- **Efficient Memory Management for LLM Serving with PagedAttention** (Kwon et al., UC Berkeley, 2023) — 提出 PagedAttention，将虚拟内存分页引入 KV Cache 管理 — https://arxiv.org/abs/2309.06180
- **DeepSeek-V2 Technical Report** (DeepSeek-AI, 2024) — 提出 MLA，低秩潜在空间压缩 KV Cache — https://arxiv.org/abs/2405.04434
- **KVQuant: Towards 10M Context Length LLM Inference** (Hooper et al., UC Berkeley, 2024) — 非均匀 KV Cache 量化方案 — https://arxiv.org/abs/2401.18079
- **H2O: Heavy-Hitter Oracle for Efficient Generative Inference** (Zhang et al., 2023) — 基于注意力分数的动态 KV 淘汰 — https://arxiv.org/abs/2306.14048
- **StreamingLLM: Efficient Streaming Language Models with Attention Sinks** (Xiao et al., MIT/Meta, 2023) — 发现 attention sink 现象，实现无限长度推理 — https://arxiv.org/abs/2309.17453